Cost-Selected Parallel Decoding of Sector–Disk Erasure Codes on CPUs and GPUs

Author Name(s)
Department, Institution
City, Country
Email address(es)

***Abstract*—Erasure-code recovery reconstructs missing storage sectors by applying finite-field equations to surviving data; these operations are costly over large regions. This paper evaluates CPU and CUDA decoders for Sector–Disk (SD) codes, which protect against complete-disk and additional sector failures. The decoders choose among normal recovery (apply matrices to data in sequence), matrix-first recovery (multiply coefficient matrices first), and partitioned recovery (solve independent groups before the coupled remainder). CPU optimization combines single-instruction multiple-data (SIMD) arithmetic with OpenMP threads; GPU variants use direct or table-based field multiplication. Across nine 32 MiB workloads, the optimized CPU achieves 1.69×–34.85× the scalar sequential baseline at eight threads. Including transfers and setup, the table-based RTX 4050 decoder is 1.32×–1.39× faster than the optimized CPU for GF(2^{32}), but slower for GF(2^8) and GF(2^{16}). CPU validation covers 7,833 feasible parameter-grid cases and 720 timed trials; separate GPU checks and all 270 CPU/GPU comparison trials pass. These results show that field arithmetic and offload overhead determine whether GPU recovery is worthwhile on the tested single-machine system.**

***Index Terms*—erasure coding, Sector–Disk codes, gPPM, OpenMP, SIMD, CUDA, recovery**

I. Introduction

Erasure codes store data with parity so that missing sectors can be reconstructed without keeping full replicas. Recovery solves finite-field equations over the surviving data, and applying those equations to large regions can make computation a significant part of repair time. This study focuses on Sector–Disk (SD) codes, which tolerate both complete-disk failures and additional sector failures. Their parity-check equations include row-local relationships as well as equations spanning the whole stripe; this structure means some failed sectors can be recovered independently while others depend on those results [1].

Prior work provides two ways to exploit this structure. Partitioned and Parallel Matrix (PPM) recovery separates independent failure groups and processes them concurrently [2]. Generalized PPM (gPPM) also selects between two equivalent calculation orders: first applying the parity-check matrices to data regions, or first multiplying the smaller coefficient matrices [3]. Single-instruction multiple-data (SIMD) instructions accelerate the finite-field operations within each region [4].

This paper asks how much these optimizations contribute to an SD decoder and whether GPU offloading improves on an already optimized CPU. We evaluate cost-based sequence selection, SIMD kernels, and OpenMP parallelism layer by layer on nine 32 MiB workloads. We then compare the final CPU decoder with direct and table-based CUDA implementations, including transfers and setup in recovery time. The CPU layers reach 1.69×–34.85× the sequential baseline; the table GPU variant adds a 1.32×–1.39× gain over the optimized CPU only in the tested GF(2^{32}) cases. This is an implementation and evaluation of existing recovery ideas on one CPU/GPU system, rather than a new code construction or a distributed-scaling study.

II. Background and Related Work

A. SD-code recovery

An SD code has n disks, r sector rows per disk, tolerates m disk failures and s additional sector failures, and uses the finite field $GF(2^w)$ of w-bit symbols. A stripe is the nr sectors protected together; z counts rows with additional sector failures. Its parity-check matrix H has $mr + s$ rows. A valid stripe B satisfies $HB = 0$. Let F, S contain the failed and surviving columns, and B_F, B_S the corresponding data. For an invertible F,

$$B_F = F^{-1} S B_S. \tag{1}$$

The normal sequence computes SB_S before applying F^{-1}. Matrix-first computes $G = F^{-1}S$ first, then applies G to the surviving data. Both recover the same sectors, but their data-region operation counts can differ [3]. SD parity equations include both row-local and stripe-wide constraints. PPM uses the row-local structure to identify independent failure groups, then recovers the coupled remainder [2].

B. Positioning among prior optimizations

PPM introduced matrix partitioning for asymmetric codes; gPPM extended it with per-matrix sequence selection that accounts for both nonzero and non-one coefficients [2], [3]. SIMD finite-field kernels accelerate each region operation and complement this reduction in work

[4]. This study keeps the SD code and parity equations fixed, and optimizes the recovery schedule and arithmetic kernels.

III. Recovery Design

A. Cost model and selection

For a coefficient matrix M, let $u(M)$ count nonzero entries and $v(M)$ count entries greater than one. A region multiply-XOR multiplies each source symbol by a field coefficient and XORs it into a destination; coefficient one requires only XOR. The estimated cost is

$$C(M) = u(M) + c\,v(M), \tag{2}$$

where c estimates the additional relative cost of a non-one multiply-XOR compared with a coefficient-one XOR. For the normal and matrix-first sequences,

$$C_{\text{normal}} = u(S) + u(F^{-1}) + c[v(S) + v(F^{-1})], \tag{3}$$
$$C_{\text{matrix}} = u(F^{-1}S) + c\,v(F^{-1}S). \tag{4}$$

For PPM, the decoder sums the cheaper sequence estimate for each independent group and the remainder. It uses this partition only when its estimated cost is lower than the cheaper whole-matrix sequence. The model estimates arithmetic work, not scheduling overhead or elapsed time.

The decoder estimates c per field and build by timing coefficient-one and coefficient-two operations on 256 KiB regions. It uses the multiply-XOR/XOR time ratio minus one, clamped at zero. Since this calibration is hardware-dependent, c can change sequence selection across platforms.

B. Partitioned and parallel execution

Rows with exactly m failed sectors have enough row-local equations to recover them independently. The decoder prepares these groups and a remainder containing the other row-local and all stripe-wide equations. OpenMP workers recover disjoint groups; a barrier ensures those outputs are ready before the remainder is decoded. The team size is capped at the number of independent groups.

The final optimization also splits the remainder's data regions into symbol-aligned byte ranges. Workers apply the same matrix-vector transformation to disjoint output ranges, avoiding write conflicts.

C. SIMD region kernels

The SIMD build dispatches at runtime with a scalar fallback. GF(2^8) uses SSSE3 byte shuffles and nibble tables (16 bytes per vector); GF(2^{16}) and GF(2^{32}) use AVX2 gathers and byte-contribution tables (eight symbols per vector). The scalar comparison uses the same M3 code with SIMD disabled.

TABLE I
CPU decoder layers in the performance comparison.

Layer	Decoder configuration
L0	Scalar normal recovery; single-threaded reference
L1	Scalar cost selection; parallel groups, serial remainder
L2	L1 with SIMD region kernels
L3	L2 with parallel byte ranges in the remainder

D. CUDA recovery

The GPU decoder retains matrix preparation and sequence selection on the host. Each CUDA thread computes one output field symbol, accumulating all nonzero coefficient contributions in a register. Adjacent threads process adjacent symbols, and one kernel evaluates a matrix-vector product rather than launching a kernel for each coefficient. Normal recovery needs two products; matrix-first needs one. Independent groups use separate streams and scratch buffers, and finish before the remainder starts. Each recovery transfers the stripe to the device once and copies the result back once.

The GPU scalar variant uses direct shift-and-reduce field multiplication. The table variant uses logarithm/antilogarithm tables for GF(2^8) and GF(2^{16}), and split-byte tables for GF(2^{32}). Both execute many CUDA threads; "scalar" describes the reference arithmetic, not serial GPU execution. Each variant calibrates its own cost weight using sixteen interleaved, separately launched XOR and multiply-XOR probes after warm-up, each on 32,768 symbols. Selection therefore uses GPU measurements rather than the CPU's cost weight.

IV. Experimental Method

A. Platform and workloads

Measurements use an Intel Core Ultra 9 185H and GCC 11.4.0 with `-O2 -fopenmp`. We test all nine $(m, s) \in \{1, 2, 3\}^2$ pairs at $n = r = 16$, $z = 1$, on a 32 MiB stripe (256 sectors of 128 KiB). For $s = 1, 2, 3$, the fields are GF(2^8), GF(2^{16}), and GF(2^{32}). Each configuration has one warm-up and ten timed recovery trials; fault injection, validation, checksums, and CSV output are excluded.

GPU tests use an RTX 4050 Laptop GPU with 6 GB memory, driver 580.178.04, and CUDA 13.0.88 compiled with `-O2`. The CPU/GPU comparison repeats the same workloads with the optimized CPU (L3, eight threads), GPU scalar, and GPU table variants, in that order. All share the same encoded bytes and failure layout. Its CPU timings are measured within this separate experiment rather than reused from the CPU-layer study.

Table I defines the four CPU layers; T is the requested OpenMP thread count. L0–L1 use the scalar build; L2–L3 use the SIMD build, so L2/L1 compares complete decoders. Cumulative speedup is the L0 median divided by a layer median; incremental gain is the previous layer's median divided by the current layer's.

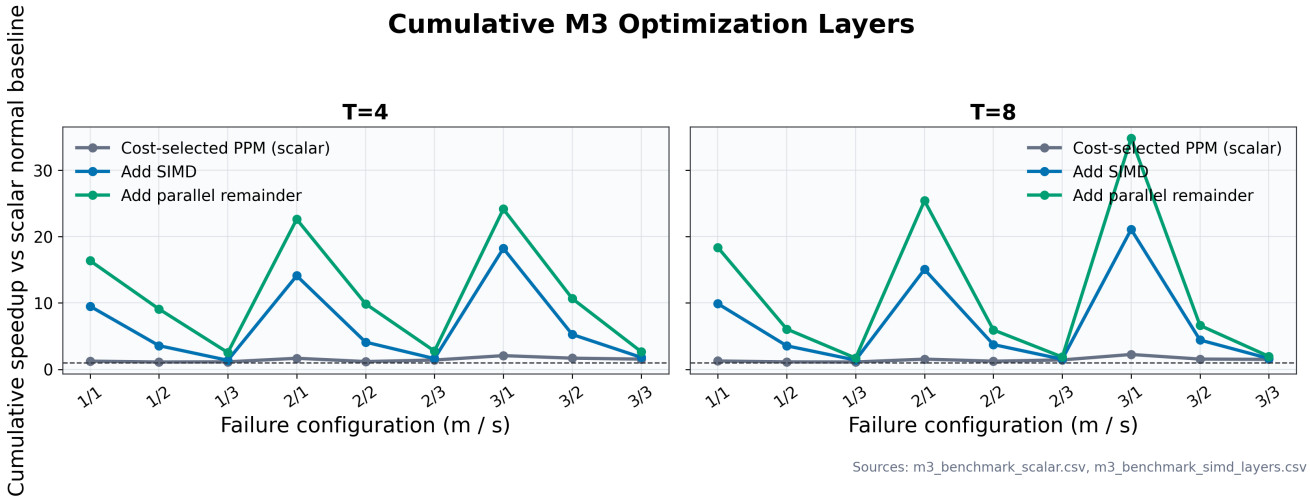


Fig. 1. Cumulative median speedup at T=4 and T=8 relative to scalar normal recovery. Curves show cost-selected scalar PPM, SIMD PPM, and SIMD PPM with a parallel remainder.

TABLE II
T=8 MEDIAN BASELINE RECOVERY TIME AND CUMULATIVE SPEEDUP AT EACH OPTIMIZATION LAYER.

m	s	Field	L0 (ms)	L1/L0	L2/L0	L3/L0
1	1	GF(2^8)	36.031	1.25×	9.89×	**18.37**×
1	2	GF(2^{16})	55.211	1.11×	3.54×	**6.07**×
1	3	GF(2^{32})	45.913	1.10×	1.38×	**1.69**×
2	1	GF(2^8)	65.968	1.53×	15.06×	**25.42**×
2	2	GF(2^{16})	82.134	1.22×	3.73×	**5.94**×
2	3	GF(2^{32})	60.146	1.41×	1.51×	**1.85**×
3	1	GF(2^8)	99.343	2.23×	21.10×	**34.85**×
3	2	GF(2^{16})	111.524	1.54×	4.41×	**6.61**×
3	3	GF(2^{32})	74.373	1.49×	1.63×	**1.95**×

B. Correctness and reported metrics

Each trial must match the saved bytes and checksum, have zero syndrome, and report the operation count implied by the selected matrices. The full-grid sweep tests one deterministic failure layout for every feasible published SD parameter and z point; it does not enumerate every possible failure placement.

Region-operation counts measure algorithmic work; elapsed time includes implementation and scheduling overhead. Useful-data throughput is non-parity bytes divided by recovery time, and speedups use median end-to-end times.

GPU end-to-end timing includes host matrix preparation, device allocation, table and stripe transfers, launches, synchronization, and cleanup. Buffers and tables are recreated per recovery; CUDA startup and cost calibration occur during warm-up. Summed per-kernel CUDA-event durations are also recorded. Because streams overlap, that sum may exceed wall-clock recovery time and cannot be subtracted from it to estimate transfer time.

V. RESULTS

A. Correctness

All 7,833 feasible CPU parameter-grid points pass; 105 of 7,938 parameter/z combinations are infeasible. At $T = 1$ and $T = 4$, recovered bytes, syndromes, and operation counts agree. All 270 scalar-build and 450 SIMD-build trials pass validation. Separate GPU tests pass 12 direct/two-stage matrix checks and 64 small-stripe recoveries across all fields, both variants, and feasible failure layouts at $n = r = 4$ and 24. NVIDIA Compute Sanitizer reports zero memory errors. All 270 timed CPU/GPU trials pass byte equality, checksum, and zero-syndrome checks, with logical operation counts stable across trials. The GPU tests do not reproduce the full CPU parameter-grid sweep.

B. Layered performance

Fig. 1 compares the optimization layers; Table II gives their cumulative T=8 speedups and baseline times.

L1 improves all nine cases by 1.10×–2.23× over L0. SIMD gives its largest gain in GF(2^8): L2 reaches 21.10× over L0 and up to 9.85× over L1. In GF(2^{32}), L2 improves on L1 by 1.07×–1.26×. Parallel remainder processing adds

TABLE III
MEDIAN END-TO-END RECOVERY TIME (MS), INCLUDING GPU TRANSFERS AND SETUP. SPEEDUP IS CPU L3 TIME DIVIDED BY GPU TABLE TIME.

m	w	CPU L3	GPU scalar	GPU table	Speedup
1	8	2.359	22.451	20.813	0.11×
1	16	8.887	21.428	27.386	0.32×
1	32	26.170	22.253	19.870	**1.32**×
2	8	2.866	35.754	29.543	0.10×
2	16	14.086	30.371	36.268	0.39×
2	32	32.032	29.426	23.980	**1.34**×
3	8	3.107	45.353	37.567	0.08×
3	16	18.416	36.269	40.136	0.46×
3	32	38.682	33.979	27.915	**1.39**×

1.20×–1.86× over L2 in every T=8 case. Final speedups range from 1.69× to 34.85×.

C. Selection behavior and scaling

M2 PPM yielded 1.02×–1.24× at $T = 1$ and up to 2.33× at $T = 8$; three cases slowed at eight threads. M3 remainder parallelism helps all T=8 cases but makes $(m, s) = (3, 1)$ about 6% slower at T=4.

D. GPU comparison

Table III reports the separate end-to-end CPU/GPU experiment. The table GPU variant is 1.32×–1.39× faster than optimized CPU L3 in GF(2^{32}), but loses to it in every GF(2^8) and GF(2^{16}) case. Table-based recovery improves on GPU scalar in six configurations, by 1.08×–1.23×; in GF(2^{16}), it instead slows recovery by factors of 1.11–1.28. The name "table" identifies the optimization technique, not a guaranteed speedup.

The full stripe moves across the host/device boundary even when recovery work is small. With per-call setup and fast CPU SIMD arithmetic, GPU offload does not pay off for the smaller fields in these workloads. GF(2^{32}) leaves more room to improve on the CPU implementation. These measurements establish that pattern, but do not isolate transfer, table-access, and scheduling costs. All timed variants selected partitioned recovery; the arithmetic cost model does not guarantee the fastest parallel schedule.

For GF(2^{32}) at $m = 1, 2, 3$, median summed kernel-event durations are 21.369, 59.978, and 83.212 ms for GPU scalar, versus 17.810, 35.989, and 53.776 ms for GPU table. These aggregate stream intervals describe kernel work, not the end-to-end latency in Table III.

VI. Scope and Future Work

The evaluation covers one CPU/GPU system, 32 MiB stripes, and nine timing configurations; the CPU sweep uses one failure layout per parameter point, and GPU coverage is narrower. It does not measure distributed scaling, storage I/O, energy, or other codes. Future work should broaden platforms and failure placements, reuse GPU buffers and tables across recoveries, and test larger or batched workloads before evaluating distributed execution.

VII. Conclusion

Cost-selected recovery, SIMD arithmetic, and OpenMP parallelism together reach 1.69×–34.85× the scalar CPU baseline across nine 32 MiB SD workloads. The CUDA comparison shows that offloading is not uniformly beneficial: table-based GPU recovery beats the optimized CPU by 1.32×–1.39× for GF(2^{32}), while CPU SIMD remains faster for the smaller fields. Correctness checks pass for the tested CPU and GPU cases. The main finding is that reducing algebraic work, accelerating arithmetic, and managing execution overhead must be considered together; more GPU threads alone do not ensure faster recovery.

Reproducibility

CPU commands are `make m3-sweep` and `make m3-benchmark`; CUDA commands are `make gpu-test`, `make gpu-benchmark`, and `make gpu-plots`. Raw CPU timings are in results/m3_benchmark_scalar.csv and results/m3_benchmark_simd_layers.csv; GPU comparison trials are in results/milestone_gpu/gpu_benchmark.csv.

References

[1] J. S. Plank, M. Blaum, and J. L. Hafner, "SD codes: Erasure codes designed for how storage systems really fail," in *Proc. 11th USENIX Conf. File and Storage Technologies (FAST)*, 2013.

[2] S. Li, Q. Cao, S. Wan, W. Zhang, C. Xie, X. He, and P. Subedi, "PPM: A partitioned and parallel matrix algorithm to accelerate encoding/decoding process of asymmetric parity erasure codes," in *Proc. 44th Int. Conf. Parallel Processing (ICPP)*, 2015.

[3] S. Li, Q. Cao, S. Wan, W. Xia, and C. Xie, "gPPM: A generalized matrix operation and parallel algorithm to accelerate the encoding/decoding process of erasure codes," *ACM Trans. Archit. Code Optim.*, vol. 20, no. 4, Art. no. 51, Dec. 2023, doi: 10.1145/3625005.

[4] J. S. Plank, K. M. Greenan, and E. L. Miller, "Screaming fast Galois field arithmetic using Intel SIMD instructions," in *Proc. 11th USENIX Conf. File and Storage Technologies (FAST)*, 2013.